

Grupos de experiencia recepcional

1. Modelos de Base de Datos Creados y Actualizados

Se implementaron las entidades del diagrama entidad-relación relativas a Cursos de ER y se vincularon con Sequelize:

- `src/models/CursoER.js`:
 - Mapea la tabla `CursoER` con `Id_Curso` (PK autoincremental), `NRC` (único) y `Nombre`.
 - `src/models/AcademicoCursoER.js`:
 - Mapea la tabla `Academico_CursoER` asociando al profesor titular (`Academico`), con su `CursoER` (`NRC`) y el periodo correspondiente (`PeriodosEscolares`).
 - `src/models/CursoEREstudiante.js`:
 - Mapea la tabla `CursoER_Estudiante` registrando la inscripción de un Estudiante en un `CursoER` para determinado `PeriodosEscolares`.
 - **Actualización en `src/models/Academico.js`:**
 - Se agregó el atributo `Id_Carrera` (asociado a `Carrera`), necesario para saber a qué carrera pertenece el Director / Jefe de Carrera.
 - **Actualización en `src/models/Lugar.js`:**
 - Se incluyó el atributo `Estado` ('disponible', 'no_disponible') en sintonía con el diagrama y el seeder.
-

2. Lógica de Negocio y Reglas de Roles Implementadas

Se construyeron servicios modulares para gestionar y aplicar las restricciones solicitadas:

A. Detección Jerárquica del Rol del Académico (`academicoService.js`)

- **Si `Id_Rol` es nulo:** El usuario es identificado como **Profesor**.
- **Si `Id_Rol` está asignado:** Se clasifica según su cargo:
 - **Director_Facultad:** Director de la Facultad.
 - **Secretario_Facultad:** Secretario Académico / Secretaria de la Facultad.
 - **Director_Carrera:** Director o Jefe de Carrera (vinculado a su `Id_Carrera`).

B. Determinación del Periodo Actual (periodoService.js)

- obtenerPeriodoActual(): Evalúa la fecha actual contra Fecha_inicio y Fecha_fin de los periodos escolares registrados (o selecciona el periodo activo más reciente) para validar las operaciones vigentes.

C. Reglas de Visibilidad de Trabajos (trabajoService.js)

En obtenerTrabajos(filtrosBusqueda, numeroPersonal):

1. **Director de Facultad y Secretario(a) de Facultad:**

- Tienen acceso total a los trabajos recepcionales de todos los grupos y de todas las carreras de la facultad.

2. **Director / Jefe de Carrera:**

- Se fuerza automáticamente el filtro por su carrera (Id_Carrera = academico.Id_Carrera), permitiéndole ver todos los trabajos de todos los grupos y periodos de su licenciatura.

3. **Profesor (no directivo):**

- Solo puede ver los trabajos de los alumnos que pertenezcan o hayan pertenecido a alguno de sus grupos de Experiencia Recepcional (incluyendo periodos históricos pasados), así como aquellos trabajos donde el profesor sea participante directo.

D. Reglas para el Registro de Trabajos (trabajoService.js y participanteService.js)

En crearTrabajoBorrador y asignarEstudiante:

1. Si quien registra es un **Profesor**, el sistema valida que cuente con al menos un grupo de Experiencia Recepcional asignado en el **periodo escolar vigente**. Si no tiene grupo activo, se rechaza la operación informando la precondition.
2. **Validación de alumnos inscritos en el periodo actual:** Si se indican alumnos (Matricula o Matriculas), el sistema valida que cada uno esté formalmente inscrito en el grupo de ER del profesor en el **periodo escolar actual**. Si un alumno pertenece a otro grupo o a un periodo pasado, el registro se detiene indicando la causa específica.

3. Nuevos Endpoints Expuestos en la API (/api/cursos)

Se crearon src/controllers/cursoController.js y src/routes/cursoRoutes.js documentados con Swagger:

Método	Endpoint	Descripción
GET	/api/cursos/periodo-actual	Consulta el periodo escolar actualmente activo.
GET	/api/cursos/periodos	Lista todos los periodos escolares registrados.
GET	/api/cursos/mis-grupos	Consulta los grupos asignados a un profesor (con filtro soloActual=true opcional) junto con sus alumnos.
POST	/api/cursos	Registra un nuevo curso/NRC de Experiencia Recepcional.
GET	/api/cursos	Lista todos los cursos/NRCs registrados.
GET	/api/cursos/:id	Detalle de un curso con profesores y estudiantes asignados.
POST	/api/cursos/asignar-profesor	Asigna a un profesor como titular de un NRC en un periodo.
POST	/api/cursos/inscribir-estudiante	Inscribe a un estudiante en un NRC en un periodo.
POST	/api/cursos/inscribir-lote	Permite inscribir múltiples estudiantes simultáneamente a un NRC.
GET	/api/cursos/:id/estudiantes	Obtiene la lista de estudiantes inscritos en un curso/periodo.

4. Pruebas y Verificación

- Se corrigió la inicialización de variables de entorno mediante dotenv en src/config/database.js y src/app.js.
- Se verificó la carga de todos los módulos y la generación dinámica de Swagger (alcanzando 36 rutas documentadas).
- Se actualizó src/seeder/initCatalogos.js para incluir los NRCs de prueba para las 4 carreras de la FEI.

Tomos en los folios.

1. Modelo de Datos Actualizado (TrabajoRecepcional.js)

Se agregaron los campos numéricos requeridos al modelo manteniéndose la compatibilidad con el campo descriptivo Folio:

- **Tomo** (DataTypes.INTEGER, nullable): Número del libro/tomo correspondiente a la carrera (\$\ge 1\$).
 - **Numero_Folio** (DataTypes.INTEGER, nullable): Número de folio asignado dentro del libro (restringido del **1 al 100**).
 - **Folio** (DataTypes.STRING): Cadena formateada para constancias y actas (ejemplo: "Tomo 2 - Folio 98").
-

2. Nuevo Servicio Especializado (folioService.js)

Este servicio encapsula las reglas de negocio sobre los libros y folios de la FEI:

1. validarTomoYFolio({ Id_Carrera, Tomo, Numero_Folio, Id_TrabajoR }):

- Valida que Tomo ≥ 1 .
- Valida estrictamente que $1 \leq \text{Numero_Folio} \leq 100$.
- **Garantiza la unicidad por carrera:** Verifica que en esa carrera (Id_Carrera) no exista otro trabajo recepcional finalizado con el mismo Tomo y Numero_Folio. Si ya está ocupado, arroja un error descriptivo indicando el trabajo que lo ocupa:

"El Folio 98 del Tomo 2 ya se encuentra ocupado en la carrera 'Ingeniería de Software' por el trabajo recepcional ID #15 ('Sistema de Gestión')."

2. sugerirSiguienteFolio(Id_Carrera, TomoSolicitado):

- **Detección de Tomo Activo:** Identifica el tomo más alto registrado para esa carrera.
- **Incremento automático:** Si ese tomo ya alcanzó sus **100 folios**, sugiere automáticamente el siguiente tomo (**Tomo + 1, Folio 1**).
- **Relleno de huecos desfasados:** Si se pasa un Tomo específico o en el tomo activo hay folios intermedios vacíos (debido a registros no secuenciales), encuentra y sugiere el primer folio libre disponible.

3. consultarEstadoTomo(Id_Carrera, Tomo):

- Retorna el número de folios ocupados, folios disponibles (de 100), el listado de folios ya tomados y si el libro ya está completo (estaLleno: true/false).

4. listarTomosPorCarrera(Id_Carrera):

- Obtiene el resumen de todos los tomos abiertos en una carrera con su métrica de folios registrados.

3. Actualización de Casos de Uso (trabajoService.js)

• Finalización de Trabajo (finalizarTrabajo):

- Ahora acepta { Tomo, Numero_Folio, Resultado, Folio (opcional) }.
- Valida que el trabajo esté en estado Generado (acta lista).
- Valida rango (1-100) y unicidad en la carrera.
- Asigna los campos, genera el folio formateado y cambia el estado a Finalizado, asentando la acción en la bitácora.

• Edición de Trabajo (actualizarTrabajo - CU-05):

- En estados Aprobado y Generado, los campos Tomo, Numero_Folio y Resultado permanecen estrictamente **bloqueados**.
- En estado Finalizado, la Secretaria de Facultad o Directivo puede editar el Tomo y Numero_Folio, revalidando que el nuevo folio no colisione con otro trabajo.

4. Nuevos Endpoints Disponibles en la API

Se crearon y documentaron en Swagger las siguientes rutas en trabajoRoutes.js:

Método	Endpoint	Parámetros Query / Body	Descripción
GET	/api/trabajos/siguiente-folio	?Id_Carrera=1&Tomo=2 (Tomo opcional)	Sugiere el siguiente folio disponible (o nuevo tomo si llegó a 100).
GET	/api/trabajos/tomo-estado	?Id_Carrera=1&Tomo=2	Consulta ocupación del libro (folios

Método	Endpoint	Parámetros Query / Body	Descripción
			ocupados, huecos libres, si está lleno).
GET	/api/trabajos/tomos	?Id_Carrera=1	Lista todos los tomos registrados de la carrera con sus totales.
POST	/api/trabajos/:id/finalizar	{ Tomo: 2, Numero_Folio: 98, Resultado: "Aprobado por Unanimidad" }	Finaliza el trabajo recepcional asignando tomo y folio formal.

Ejemplo de Respuesta al Sugerir Folio:

Cuando la secretaria abre la ventana para finalizar un trabajo en la carrera con Id_Carrera = 1:

json

```
{
  "Id_Carrera": 1,
  "Tomo": 2,
  "Numero_Folio": 99,
  "FolioSugerido": "Tomo 2 - Folio 99",
  "foliosOcupadosEnTomo": 98,
  "foliosDisponiblesEnTomo": 2,
  "esNuevoTomo": false,
  "estaLleno": false
}
```

Y si el tomo tuviese sus 100 folios ocupados:

json

```
{
  "Id_Carrera": 1,
  "Tomo": 3,
```

```
"Numero_Folio": 1,  
"FolioSugerido": "Tomo 3 - Folio 1",  
"foliosOcupadosEnTomo": 0,  
"foliosDisponiblesEnTomo": 100,  
"esNuevoTomo": true,  
"estaLleno": false,  
"mensaje": "El Tomo 2 ha alcanzado su límite de 100 folios. Se abre automáticamente el  
Tomo 3."  
}
```